

3

Performing operations

This chapter demonstrates the various PHP operators for arithmetic, comparison and logical evaluation.

Doing arithmetic

Making comparisons

Examining conditions

Assessing logic

Comparing bits

Changing values

Grasping precedence

Summary

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus
**	Exponentiation

a = b * c - d % e / f ;	# This is unclear.
a = (b * c) - ((d % e) / f) ;	# This is clearer.



McGrath, Mike. PHP in Easy Steps, 4th Edition : Updated for PHP 8, In Easy Steps Limited, 2021. ProQuest Ebook Central, <http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docId=7075469>.
Created from univ-people-ebooks on 2026-04-18 09:27:42.

The ****** exponentiation operator gives the result of raising the first operand to the power of the second operand.

a ** b **# Result of raising a to the b'th power.**

Shorthand expressions can usefully be created by combining an arithmetic operator with the = assignment operator – for example:

a += b ; **# Equivalent to a = (a + b) ;**

a -= b ; **# Equivalent to a = (a - b) ;**

a *= b ; **# Equivalent to a = (a * b) ;**

a /= b ; **# Equivalent to a = (a / b) ;**



Group expressions with parentheses to specify evaluation precedence – innermost groups get evaluated first. See [here](#) for more on operator precedence.

- 1 Create a valid HTML document, like the one listed [here](#), then insert PHP tags into the body section

<?php

Statements to be inserted here.

?>



arithmetic.php

- 2 Now, insert statements between the PHP tags to create and initialize two variables

\$a = 5 ;

\$b = 2 ;

- 3 Next, insert statements to display the results of simple arithmetical operations using the variable values

**\$result = \$a + \$b ; echo "Addition : \$result
" ;**

**\$result = \$a - \$b ; echo "Subtraction : \$result
" ;**

```
$result = $a * $b ; echo "Multiplication : $result <br>" ;
```

```
$result = $a / $b ; echo "Division : $result <br>" ;
```

4

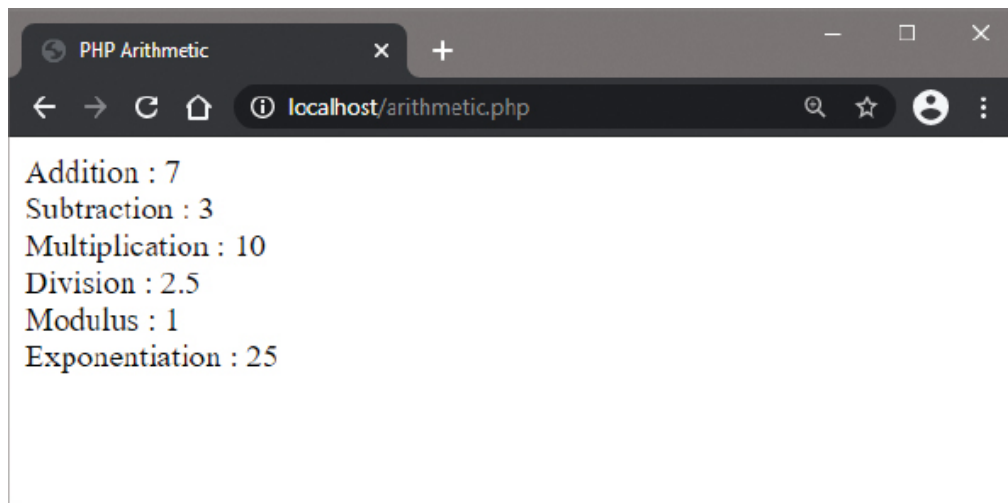
Finally, insert a statement to display the results of arithmetical evaluations of the variable values

```
$result = $a % $b ; echo "Modulus : $result <br>" ;
```

```
$result = $a ** $b ; echo "Exponentiation : $result" ;
```

5

Save the document in your web server's **/htdocs** directory as **arithmetic.php** then open the page via HTTP to see the results and variable values get displayed



PHP has some useful functions to work with numbers. For example, **round(\$a)** rounds **\$a** to the nearest integer and **number_format(\$a)** adds commas to **\$a** denoting groups of thousands.

Making comparisons



The operators that are commonly used in PHP scripts to compare two values are listed in the table below:

Operator	Comparative test
==	Equality
===	Identity
!==	Non-identity
!= <>	Inequality
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
<=>	Spaceship

The `==` equality operator compares two operands and returns a Boolean **TRUE** result if both are numerically equal in value, otherwise it returns **FALSE**. Characters and strings are also compared numerically by their ASCII code value. Conversely, the `!=` inequality operator returns **TRUE** if the two operands are not equal, otherwise it returns **FALSE**.

The `>` “greater than” operator compares two operands and returns **TRUE** if the first is greater in value than the second, or it returns **FALSE** if it is equal or less in value. The `<` “less than” operator makes the same comparison but returns **TRUE** if the first operand is less in value than the second, otherwise it returns **FALSE**. Adding the `=` operator after a `>` “greater than” or `<` “less than” operator makes it also return **TRUE** when the two operands are exactly equal.

The `<=>` spaceship operator returns an integer value of 1 when the first operand is greater than the second, -1 when the first operand is less than the second, or 0 when the operands are equal.

The PHP `var_dump()` function can be used to see the type and value resulting from each kind of comparison.



Equality and inequality operators are useful in testing the state of two variables to perform conditional branching in a PHP script.

- 1 Create a valid HTML document, like the one listed [here](#), then insert PHP tags into the body section

```
<?php
```

```
# Statements to be inserted here.
```

```
?>
```



comparison.php

- 2 Now, insert statements between the PHP tags to create and initialize five variables

```
$zero = 0 ; $nil = 0 ; $one = 1 ; $supr = 'A' ; $lwr = 'a' ;
```

- 3 Next, insert statements to display equality comparisons

```
echo "0 == 0 : " ;      var_dump( $zero == $nil ) ;
```

```
echo "<br>0 == 1 : " ;  var_dump( $zero == $one ) ;
```

```
echo "<br>A == a : " ;  var_dump( $supr == $lwr ) ;
```

```
echo "<br>A != a : " ;  var_dump( $supr != $lwr ) ;
```

- 4 Insert statements to display comparison evaluations

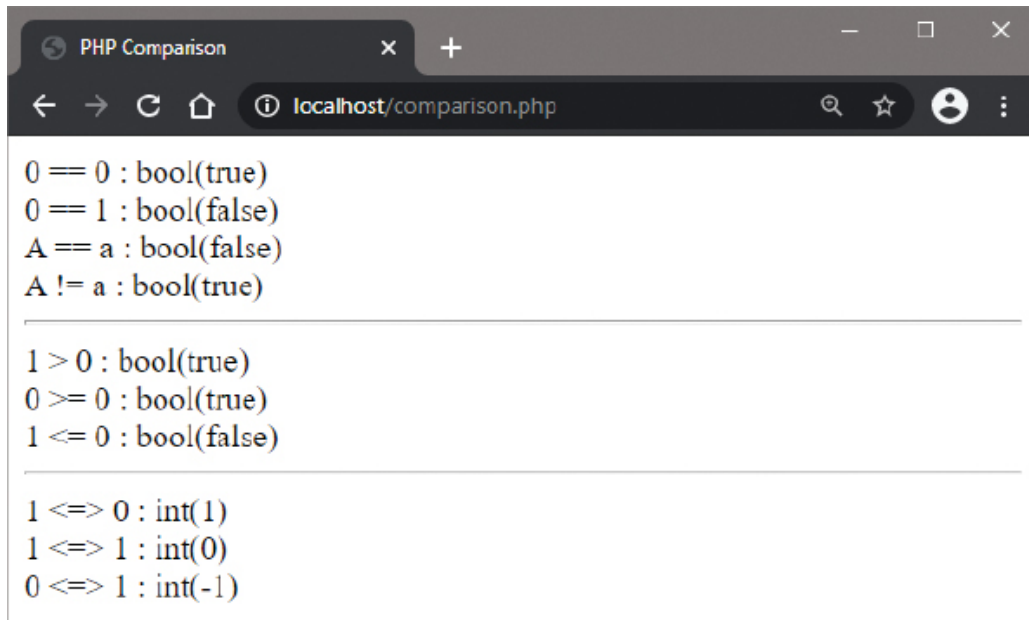
```
echo "<hr>1 > 0 : " ;   var_dump( $one > $nil ) ;
```

```
echo "<br>0 >= 0 : " ;  var_dump( $zero >= $nil ) ;
```

```
echo "<br>1 <= 0 : " ;  var_dump( $one <= $nil ) ;
```

```
echo "<hr>1 <=> 0 : “; var_dump( $one <=> $nil );  
echo "<br>1 <=> 1 : “; var_dump( $one <=> $one );  
echo "<br>0 <=> 1 : “; var_dump( $nil <=> $one );
```

5 Save the document in your web server's **/htdocs** directory as **comparison.php** then open the page via HTTP to see the results get displayed



```
0 == 0 : bool(true)  
0 == 1 : bool(false)  
A == a : bool(false)  
A != a : bool(true)  
  
1 > 0 : bool(true)  
0 >= 0 : bool(true)  
1 <= 0 : bool(false)  
  
1 <=> 0 : int(1)  
1 <=> 1 : int(0)  
0 <=> 1 : int(-1)
```



The ASCII code value for uppercase ‘**A**’ is 65 and for lowercase ‘**a**’ it’s 97 – so their comparison here returns **FALSE**.



A Boolean **TRUE** value is represented numerically by 1 in PHP language.

Examining conditions

Conditional operator



The conditional operator **?:** is also known as the “ternary” operator as it has three components. This operator first evaluates an expression for a **TRUE** or **FALSE** Boolean value, then returns one of two specified results depending on the evaluation. The conditional operator syntax looks like this:

(*test-expression*) ? *result-if-true* : *result-if-false* ;

The conditional operator can be used to evaluate whether a given number is one or more, to ensure correct grammar in output regarding singular and plural items. This avoids awkward phrases such as “There is 5”:

```
$verb = ( $number == 1 ) ? 'is' : 'are';  
echo “There $verb $number” ;
```

In this case, when the **\$number** variable has a value of one, the **‘is’** result will be assigned to the **\$verb** variable, otherwise the **‘are’** result will be assigned.

The conditional operator can also be used to evaluate whether a given number is odd or even (“parity”) by examining if there is any remainder after dividing the number by two, then assigning an appropriate string result, like this:

```
$parity = ( $number % 2 == 0 ) ? 'Odd' : 'Even' ;  
echo “$number is $parity” ;
```

In this case, when the modulus operation returns a value of one, the **‘Odd’** result will be assigned to the **\$parity** variable, otherwise the **‘Even’** result will be assigned.



Do not use more than one `?:` conditional operator in a single statement as the results may be unpredictable.

Null coalescing operator

PHP has a special **NULL** value that is a built-in constant, which represents a variable with no value whatsoever – not even zero. Variables that have not been assigned a value will evaluate as **NULL**. The snappily named `??` “null coalescing” operator can be used to traverse a number of operands, from left to right, and return the value of the first operand that is not **NULL**. If none of the operands have a value (and are not **NULL**) then the null coalescing operator will itself return a **NULL** result.

- 1 Create a valid HTML document, like the one listed [here](#), then insert PHP tags into the body section

```
<?php
```

```
# Statements to be inserted here.
```

```
?>
```



condition.php

- 2 Now, insert statements between the PHP tags to create and initialize three variables

```
$a = NULL ; $b = 8 ; $c = 'PHP Fun' ;
```

- 3 Next, insert statements to output correct grammar

```
$verb = ( $b == 1 ) ? 'is' : 'are' ;
```

```
echo "There $verb $b <hr>";
```

- 4 Insert statements to display parity of the integer variable

```
$parity = ( $b % 2 != 0 ) ? 'Odd' : 'Even' ;
```

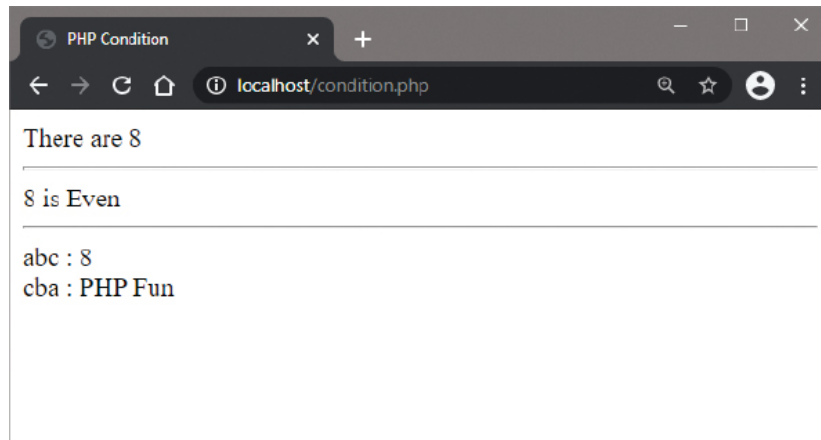
```
echo "$b is $parity <hr>";
```

- 5 Finally, insert statements to display the non-null values

```
$result = $a ?? $b ?? $c ; echo "abc : $result <br>" ;
```

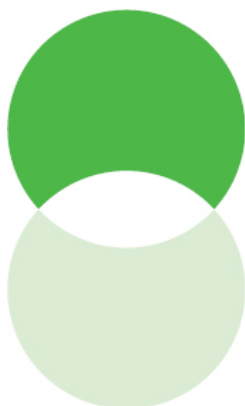
```
$result = $c ?? $b ?? $a ; echo "cba : $result <br>" ;
```

6 Save the document in your web server's **/htdocs** directory as **condition.php** then open the page via HTTP to see the results get displayed



Notice that the **NULL** value in **\$a** is ignored.

Assessing logic



The logical operators most commonly used in PHP scripts are listed in the table below:

Operator		Operation
and	&&	Logical AND
or	 	Logical OR
xor		Logical eXclusive OR
!		Logical NOT

The logical operators are used with operands that have the Boolean values of **TRUE** or **FALSE**, or an expression that can convert to **TRUE** or **FALSE**.

The **and** operator will evaluate two operands and return **TRUE**, only if both operands are themselves **TRUE**. Otherwise, the **and** operator will return **FALSE**. This is used in conditional branching where the direction of a PHP script is determined by testing two conditions. If both conditions are satisfied, the script will go in a certain direction, otherwise it will take a different direction.

Unlike the **and** operator that needs both operands to be **TRUE**, the **or** operator will evaluate two operands and return **TRUE** if either one of the operands is itself **TRUE**. If neither operand is **TRUE** then the **or** operator will return **FALSE**. This is useful to perform an action if either one of two test conditions has been met.

The **xor** operator will evaluate two operands and only return **TRUE** if either one of the operands is itself **TRUE** – but not both. If neither operand is **TRUE**, or if both operands are **TRUE**, the **xor** operator will return **FALSE**.

The **!** logical NOT operator is a “unary” operator – that is used before a single operand. It returns the inverse value of the given operand, so if the variable **\$var** is a **TRUE** value then **! \$var** would return a **FALSE** value. The **!** operator is useful to toggle the value of a variable in a loop with a statement like **\$var = ! \$var**. On each iteration the value is reversed, like flicking a switch on and off.



The **&&** operator is an alternative form of the **and** operator. Similarly, the **||** operator is an alternative form of the **or** operator.



The term “Boolean” refers to a system of logical thought developed by the English mathematician George Boole (1815-1864).

- 1 Create a valid HTML document, like the one listed [here](#), then insert PHP tags into the body section

```
<?php
```

```
# Statements to be inserted here.
```

```
?>
```



logic.php

- 2 Now, insert between the PHP tags statements to create and initialize two variables with Boolean values

```
$yes = TRUE ; $no = FALSE ;
```

- 3 Next, insert statements to display AND evaluations

```
$result = ( $no && $no ) ? 'TRUE' : 'FALSE' ;
```

```

    echo "No AND No returns $result <br>" ;

$result = ( $yes && $no ) ? 'TRUE' : 'FALSE' ;

    echo "Yes AND No returns : $result <br>" ;

$result = ( $yes && $yes ) ? 'TRUE' : 'FALSE' ;

    echo "Yes AND Yes returns $result <hr>" ;

```

4 Insert statements to display OR and NOT evaluations

```

$result = ( $no || $no ) ? 'TRUE' : 'FALSE' ;

    echo "No OR No returns $result <br>" ;

$result = ( $yes || $no ) ? 'TRUE' : 'FALSE' ;

    echo "Yes OR No returns $result <br>" ;

$result = ( $yes || $yes ) ? 'TRUE' : 'FALSE' ;

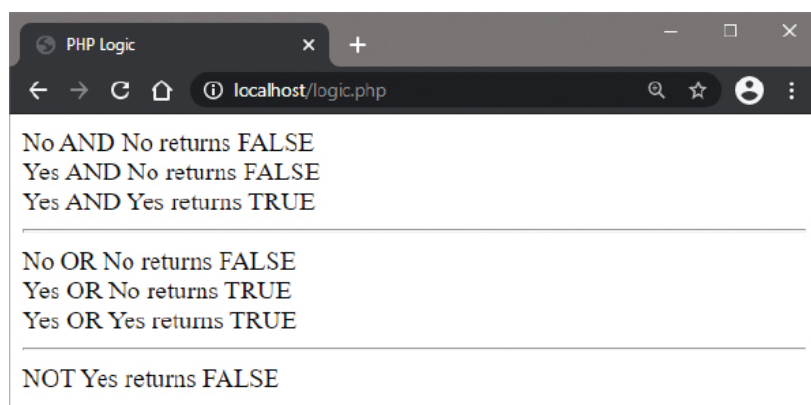
    echo "Yes OR Yes returns $result <hr>" ;

$result = ( ! $yes ) ? 'TRUE' : 'FALSE' ;

    echo "NOT Yes returns $result <br>" ;

```

5 Save the document in your web server's **/htdocs** directory as **logic.php** then open the page via HTTP to see the results get displayed



PHP also has an **XOR** (exclusive **OR**) operator that performs the equivalent evaluation of **AND NOT** together.



Notice that evaluation of **FALSE && FALSE** returns **FALSE** – maybe demonstrating the anecdote “two wrongs don’t make a right”.

Comparing bits

A byte comprises eight bits that can each contain a **1** or a **0** to store a binary number, representing decimal values from 0 to 255. Each bit contributes a decimal component only when that bit contains a **1**. Components are designated right-to-left from the “Least Significant Bit” (LSB) to the “Most Significant Bit” (MSB). The binary number in the bit pattern below represents decimal 50.



Bit No.	8	7	6	5	4	3	2	1
	MSB							LSB
Decimal	128	64	32	16	8	4	2	1
Binary	0	0	1	1	0	0	1	0

It is possible to manipulate individual parts of a byte using “bitwise” operators. Bitwise operators allow evaluation and manipulation of specific bits within an integer.

Operator	Name	Binary number operation
	OR	Return a 1 in each bit where either of two compared bits is a 1 Example: 1010 0101 = 1111
&	AND	Return a 1 in each bit where both of two compared bits is a 1 Example: 1010 & 1100 = 1000
~	NOT	Return a 1 in each bit where neither of two compared bits is a 1 Example: ~1010 = 0101
^	XOR	Return a 1 in each bit where only one of two compared bits is a 1 Example: 1010 ^ 0100 = 1110
<<	Shift left	Move each bit that is a 1 a specified number of bits to the left Example: 0010 << 2 = 1000

>>

Shift right

Move each bit that is a **1** a specified number of bits to the rightExample: **1000 >> 2 = 0010**

Each half of a byte is known as a “nibble” (4 bits). The binary Many numbers in the examples in the table describe values stored in a nibble.



Many PHP programmers never use bitwise operators, but it is useful to understand what they are and how they may be used.

Unless programming for a device with limited resources, there is seldom a need to utilize bitwise operators, but they can be useful. For instance, the ^ (eXclusive OR) operator lets you exchange values between two variables without the need for a third variable:

- 1 Create a valid HTML document, like the one listed [here](#), then insert PHP tags into the body section

<?php

Statements to be inserted here.

?>



bitwise.php

- 2 Now, insert between the PHP tags statements to create and initialize two variables with integer values

\$x = 5 ; \$y =10 ;

- 3 Next, add a statement to display the assigned values

echo “X : \$x , Y : \$y
” ;

4 Insert three XOR statements to exchange the variable values by binary bit manipulation

```
$x = $x ^ $y ;
```

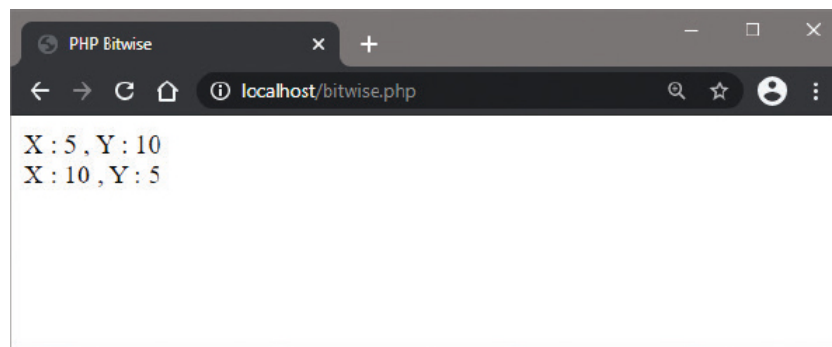
```
$y = $x ^ $y ;
```

```
$x = $x ^ $y ;
```

5 Finally, add a statement to display the manipulated values

```
echo "X : $x , Y : $y <br>" ;
```

6 Save the document in your web server's **/htdocs** directory as **bitwise.php** then open the page via HTTP to see the XOR bitwise magic



Do not confuse bitwise operators with logical operators. Bitwise operators compare binary numbers, whereas logical operators evaluate Boolean values.

Changing values



The PHP ++ increment operator and -- decrement operator alter the given number by 1 and return the resulting new value. These are most often used to count iterations in a loop. The increment operator increases the value by 1 and the decrement operator decreases the value by 1. Each of these operators may appear either before or after the variable containing the number to be changed – to different effect! When either of these operators is placed before the variable (“prefixed”), they change the value by 1 then return the modified value, but when they are placed after the variable (“postfixed”) they return the current value, then change the value by 1:

- 1 Create a valid HTML document, like the one listed [here](#), then insert PHP tags into the body section

```
<?php
```

```
# Statements to be inserted here.
```

```
?>
```



change.php

- 2 Now, insert between the PHP tags statements to create and initialize four variables with the same integer value

```
$a = $b = $c = $d = 5 ;
```

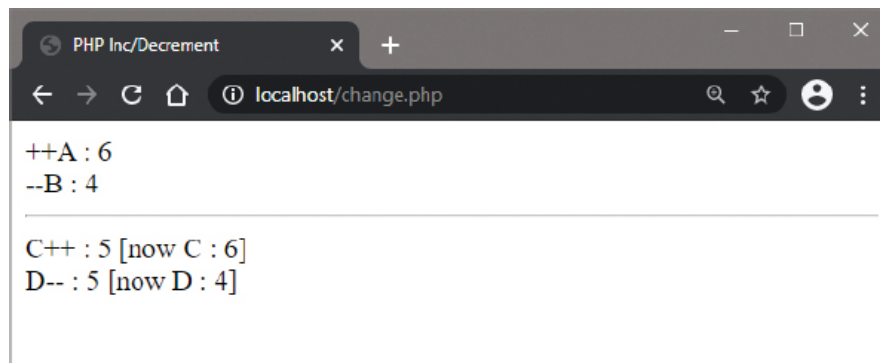
- 3 Next, add statements to change each of the values by 1 and display the new integers

```
echo "++A : ". ++$a . "<br>--B : ". --$b . "<br>" ;
```

```
echo "C++ : ". $c++ . "[now C : ". $c . "]"<br>" ;
```

```
echo "D-- : ". $d-- . "[now D : ". $d . "]"<br>" ;
```

4 Save the document in your web server's **/htdocs** directory as **change.php** then open the page via HTTP to see the results



```
++A : 6
--B : 4

C++ : 5 [now C : 6]
D-- : 5 [now D : 4]
```



Notice how the `.` concatenation operator is used here to form strings. You can also use the `.=` concatenating assignment operator to append one string onto another string.

Grasping precedence

PHP operator precedence determines how expressions are grouped for evaluation. For example, the expression `1 + 5 * 3` evaluates to 16, not 18, because the `*` multiplication operator has a higher precedence than the `+` addition operator. Parentheses can be used to specify precedence, so that `(1 + 5) * 3` evaluates to 18. When operators have equal precedence, their “associativity” determines how expressions are grouped. For example, the `-` subtraction operator is left-associative, so `8 - 4 - 2` is grouped as `(8 - 4) - 2` and evaluates to 2. The table below lists common operators in order of precedence, with the highest-precedence ones at the top. Operators on the same line have equal precedence, so operator associativity determines how expressions are grouped.



Operator	Associativity
<code>**</code>	Right
<code>++ -- ~</code>	Right
<code>!</code>	Right
<code>* / %</code>	Left
<code>+ - .</code>	Left
<code><< >></code>	Left
<code>< <= > >=</code>	None
<code>== != === !== <> <=></code>	None
<code>&</code>	Left
<code>^</code>	Left
<code> </code>	Left

&&	Left
 	Left
??	Right
?=	Left
=+= -= *= /= **= .= %= &= = ^= <<= >>= ==>	Right
and	Left
xor	Left
or	Left
,	Left



The eagle-eyed will notice that the alternative forms of logical operators have different levels of precedence.

This can cause unexpected results in an expression such as **\$bool = true and false** versus the alternative **\$bool = true && false**. Try each of these then do **var_dump(\$bool)** to see the difference.

Summary

- Arithmetical operators can form expressions with operands for addition **+**, subtraction **-**, multiplication *****, division **/**, modulus **%**, and exponentiation ******.
- The assignment operator **=** can be combined with any arithmetical operator to perform an arithmetical calculation then assign the result.
- Comparison operators can form expressions with operands for equality **==**, inequality **!=**, identity or not **===**, **!==**, greater **>**, less **<**, greater, less or equal **>=**, **<=**, **<=>**.
- The conditional operator **?:** evaluates an expression for a **TRUE** or **FALSE** Boolean value, then returns one of two specified results depending on the evaluation.
- The **??** null coalescing operator traverses operands from left to right and returns the first value that is not **null**.
- Logical operators **and**, **&&**, **or**, **||** and **xor** form expressions evaluating two operands and return a value of **TRUE** or **FALSE**.
- The logical **!** not operator returns the inverse Boolean value of a single operand.
- Bitwise operators **|**, **&**, **~**, **^**, **<<** and **>>** allow the evaluation and manipulation of specific bits within an integer.
- Increment **++** or decrement **--** operators that prefix a variable change its value by 1 and return the result.
- Increment **++** or decrement **--** operators that postfix a variable return its current value, then change its value by 1.
- Operator precedence determines how expressions are grouped for evaluation.
- When operators have equal precedence, their associativity determines how the expression is grouped for evaluation.
- The **var_dump()** function can be used to see the type and value resulting from an expression.
- The special **NULL** constant represents a variable that has no value whatsoever, not even zero.